Recursion Survival Guide

Just enough recursion for mergesort, quicksort, and trees

A straightforward recursion example

```
1  public static long factorial(int n) {
2      if (n <= 1) return 1;              // base case
3      return n * factorial(n - 1);       // closer to the base case
4  }
```

A **recursive** method calls itself on a **smaller** version of the same problem.

Every recursive method has the same **two parts**. Once you can spot them, mergesort's recursion is much less scary.

The two parts of every recursive method

1. **A base case, with a trivial solution.**
   The smallest version of the problem, solved directly, with no recursion. For a sum: an empty range adds up to 0.

2. **A recursive call that always gets closer to the base case.**
   Call the method on a smaller version of the problem. If a call doesn't move toward the base case, the calls never stop.

When you write the recursive call: trust it.

Assume it returns the right answer for the smaller problem, and just use that answer. (Writing what the method returns as a comment first tells you exactly what you're trusting it to do.)

Example: the sum of a[lo..hi]

```
1  // Returns the sum of a[lo..hi].
2  public static int sum(int[] a, int lo, int hi) {
3      if (hi < lo) return 0;                    // empty: sum is 0
4      return a[lo] + sum(a, lo + 1, hi);        // first + the rest
5  }
```

1. Base case: an empty range (`hi < lo`) adds up to 0.

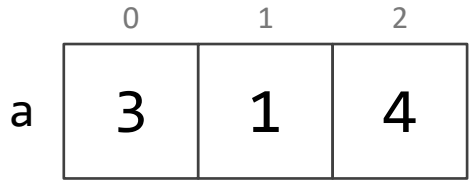
2. Recursive call: `sum(a, lo + 1, hi)` starts one item later, so every call gets **closer to the empty range**.

Trust it: the comment on line 1 says what `sum` returns. Assume the call returns the sum of the rest, then add `a[lo]`.

What Java actually does: the call stack

```
sum(a, 0, 2)
```

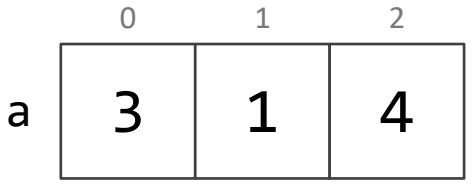
calls sum(a, 1, 2)



Each call **waits** for the call below it to answer.

What Java actually does: the call stack

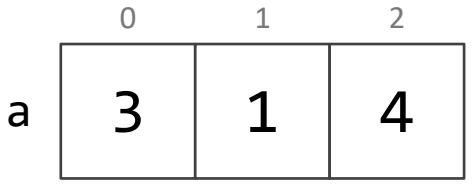
`sum(a, 0, 2)`	waiting...
`sum(a, 1, 2)`	**calls sum(a, 2, 2)**



Each call **waits** for the call below it to answer.

What Java actually does: the call stack

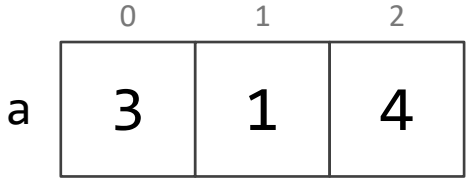
`sum(a, 0, 2)`	waiting...
`sum(a, 1, 2)`	waiting...
`sum(a, 2, 2)`	**calls sum(a, 3, 2)**



Each call **waits** for the call below it to answer.

What Java actually does: the call stack

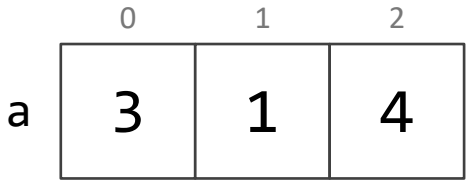
`sum(a, 0, 2)`	waiting...
`sum(a, 1, 2)`	waiting...
`sum(a, 2, 2)`	waiting...
`sum(a, 3, 2)`	**empty range: return 0**



Each call **waits** for the call below it to answer.

What Java actually does: the call stack

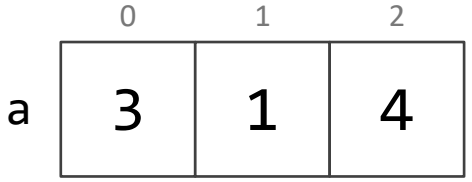
`sum(a, 0, 2)`	waiting...
`sum(a, 1, 2)`	waiting...
`sum(a, 2, 2)`	waiting...
`sum(a, 3, 2)`	**returns 0**



Answers come back **up** the stack, one at a time.

What Java actually does: the call stack

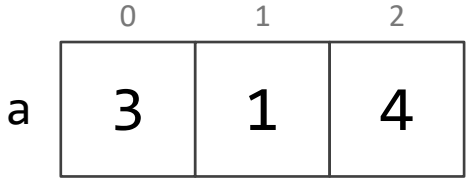
`sum(a, 0, 2)`	waiting...
`sum(a, 1, 2)`	waiting...
`sum(a, 2, 2)`	**returns 4 (4 + 0)**



Answers come back **up** the stack, one at a time.

What Java actually does: the call stack

`sum(a, 0, 2)`	waiting...
`sum(a, 1, 2)`	**returns 5 (1 + 4)**

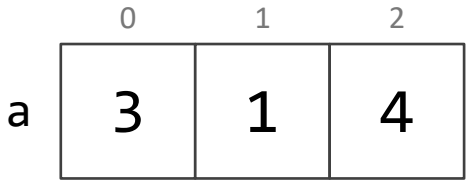


Answers come back **up** the stack, one at a time.

What Java actually does: the call stack

```
sum(a, 0, 2)
```

returns 8 (3 + 5)



Answers come back **up** the stack, one at a time.

sum = 8. Each call only ever did one small thing: add one item to an answer it trusted.

You don't have to trace it to write it

Tracing every call is useful to see **once**. You don't need it to **write** a recursive method. You only need to ask:

- Is the **base case** right?
- Does every recursive call get **closer to the base case**?
- **If** the recursive calls return the right answers, does my method return the right answer?

If all three answers are yes, the method works, at every size.

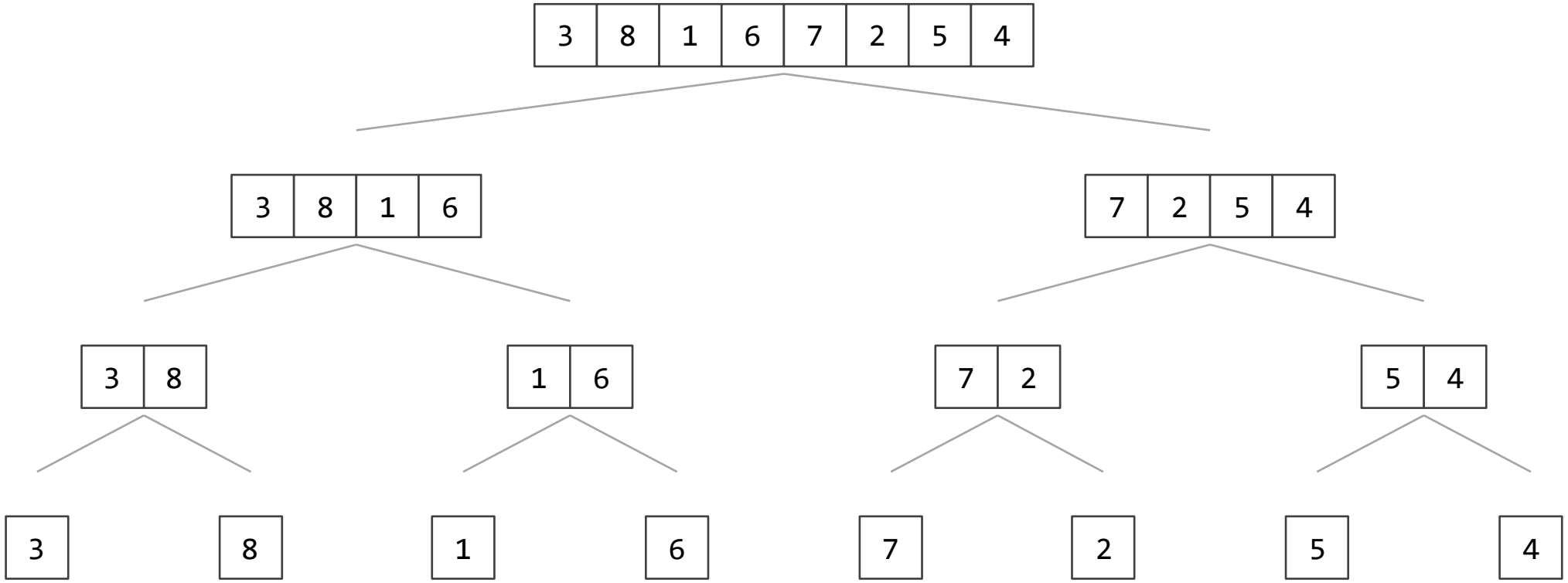
Trace by hand when you're **debugging**, not when you're writing.

Example 2: max, by splitting in half

```
1  // Returns the largest item in a[lo..hi].
2  public static int max(int[] a, int lo, int hi) {
3      if (hi == lo) return a[lo];              // one item: it's the max
4      int mid = lo + (hi - lo) / 2;
5      int left  = max(a, lo, mid);             // max of the left half
6      int right = max(a, mid + 1, hi);         // max of the right half
7      return Math.max(left, right);            // combine
8  }
```

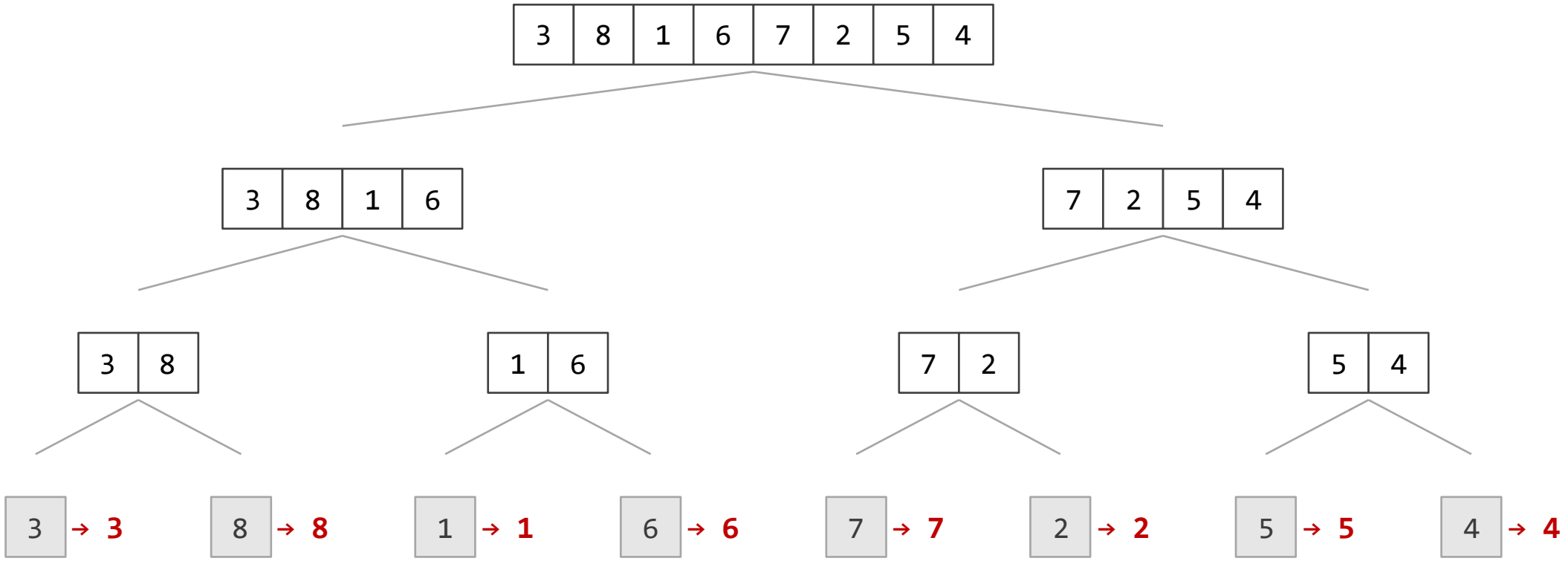
Two recursive calls this time. That's fine: each one is on **half the range**, so each one gets **closer to the base case**. Trust both of them.

max: the calls



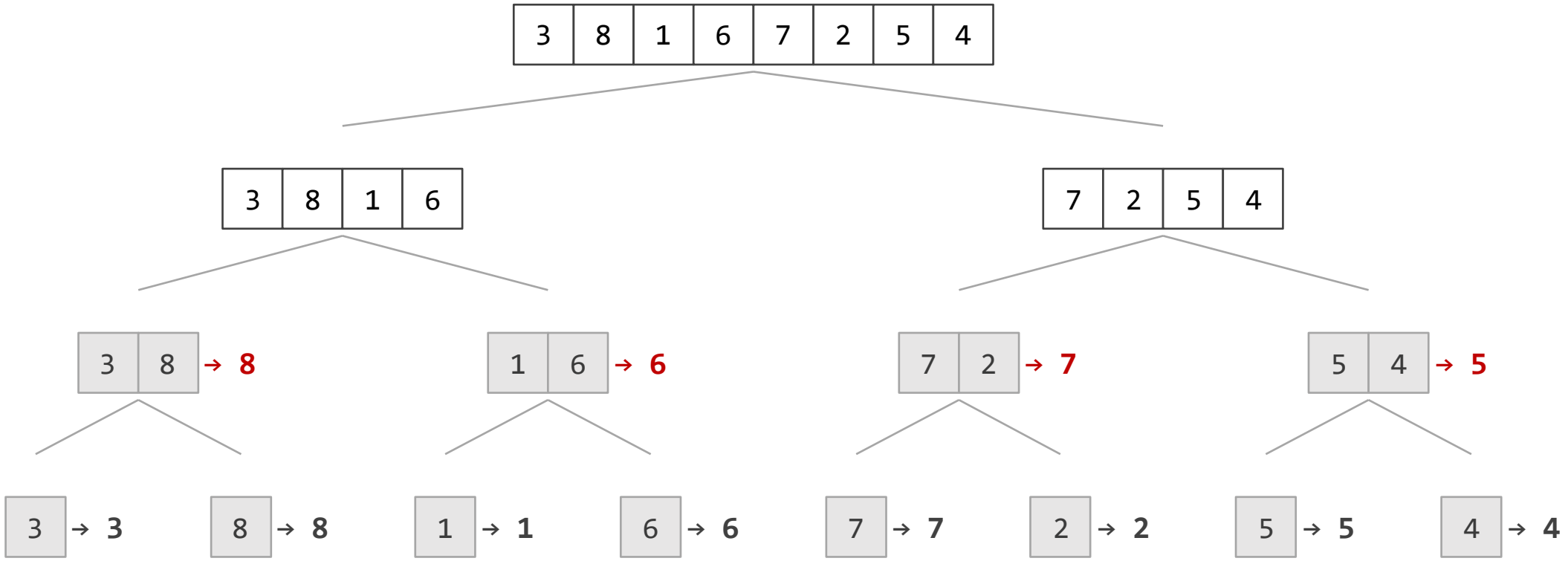
Each box is one call. Ranges are cut in half until each call has **one item**.

max: the calls



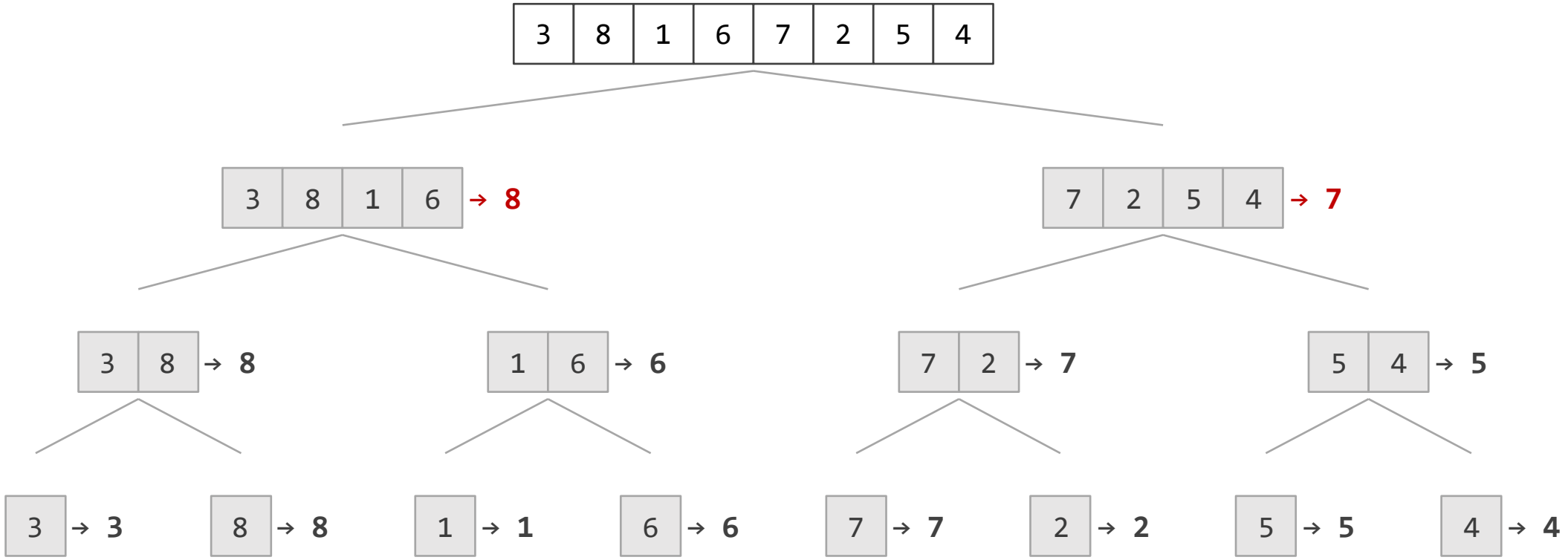
One-item calls return right away.

max: the calls



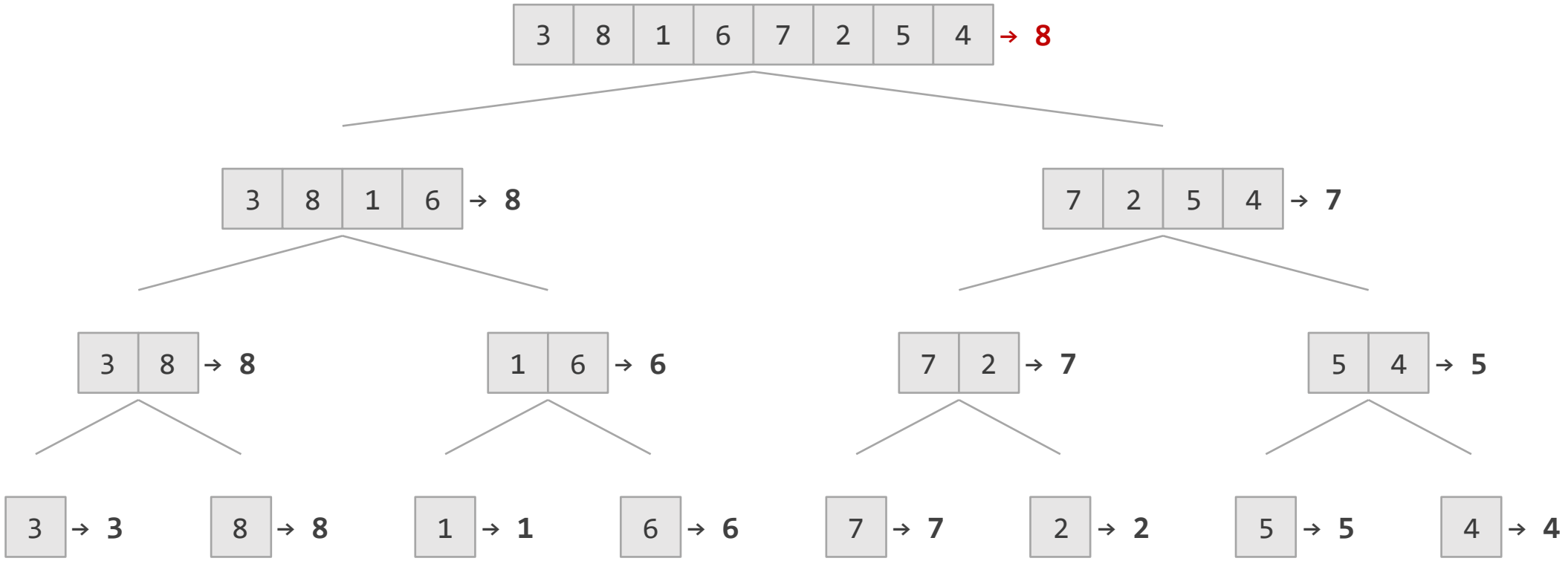
Each call returns the larger of its two halves' answers.

max: the calls



And again, one level up.

max: the calls



The top call returns **8**. Split all the way down, then **combine on the way back up**.

Same shape as mergesort

max

```
1  max(a, lo, hi):
2      if hi == lo: return a[lo]
3      mid = lo + (hi - lo) / 2
4      left  = max(a, lo, mid)
5      right = max(a, mid + 1, hi)
6      return the larger of left and right
```

mergesort

```
1  sort(a, aux, lo, hi):
2      if hi <= lo: return
3      mid = lo + (hi - lo) / 2
4      sort(a, aux, lo, mid)
5      sort(a, aux, mid + 1, hi)
6      merge(a, aux, lo, mid, hi)
```

Lines 2–5 are the **same shape**: base case, find the middle, recurse on `lo..mid`, recurse on `mid+1..hi`.

Only the last line differs: max **combines** two answers by picking the larger. Mergesort **combines** two sorted halves by merging them.

When recursion goes wrong

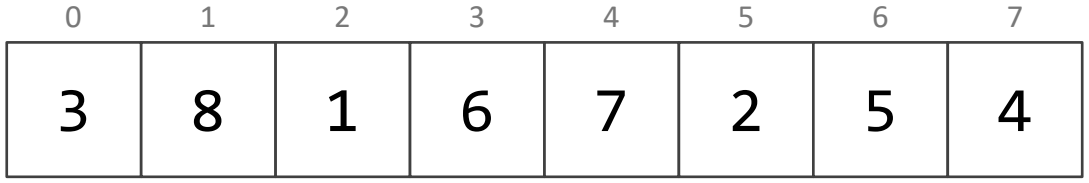
mistake	what you'll see
no base case, or one that's never reached	`StackOverflowError`
a recursive call that doesn't get closer to the base case (for example, mid instead of mid + 1)	`StackOverflowError`
calling recursively but ignoring the answer it returns	a wrong answer, with no error

A **StackOverflowError** means the calls never stopped: every call made another one, until Java ran out of room to keep track of them.

Try it: min

Pause the video, and write a recursive method that returns the **smallest** item in `a[lo..hi]`, by splitting in half, the same way `max` does.

Find the two parts: what's the base case? What are the recursive calls, and how do they get closer to it? Then: what do you do with the two answers?



Try it: min (answer)

```
1  // Returns the smallest item in a[lo..hi].
2  public static int min(int[] a, int lo, int hi) {
3      if (hi == lo) return a[lo];
4      int mid = lo + (hi - lo) / 2;
5      int left  = min(a, lo, mid);
6      int right = min(a, mid + 1, hi);
7      return Math.min(left, right);
8  }
```

It's `max` with the names changed. If yours looks like this, **you're ready for mergesort's recursion.**

The two parts, one more time

1. A **base case**, with a trivial solution.

2. A **recursive call** that always gets **closer to the base case**.

When you write the recursive call, **trust it**: assume it returns the right answer, and use that answer.

You'll see these two parts in **mergesort** this week, **quicksort** next week, and **binary search trees** after that.